

STAT 547 - Data Mining

Homework 2

Rochester Institute of Technology
Ducheng Tan

March 15, 2026

Exercise 1: High-Dimensional Classification (40 points)

Consider a binary classification task with a dataset $D_n = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^p$, $y_i \in \{0, 1\}$, drawn from an unknown joint distribution $p_{xy}(x, y)$.

A. Theoretical Setup

1. Derive the explicit form of the decision boundary for Linear Discriminant Analysis (LDA) in the binary case, starting from the discriminant function:

$$\delta_g(x) = x^\top \Sigma^{-1} \mu_g - \frac{1}{2} \mu_g^\top \Sigma^{-1} \mu_g + \log(\pi_g). \quad (1)$$

We must note that (1) is a proxy to the the Bayes classifier for classification, in this case we have $\delta_g(x)$ to estimate $P(Y = g|X = x)$. Then when classifying x we must assign x to class g_0 iff

$$P(Y = g_0|X) > P(Y = g_i|X), \quad 0 < i \leq G \quad (2)$$

Interpret the boundary in terms of the Mahalanobis distance. In practice the Bayes conditional is not accessible due to intractable integrals and unknown joint probability density functions. Hence we let the decision boundary for LDA be defined as the set of points x where the two discriminant functions are equal, i.e. $\delta_1(x) = \delta_0(x)$. Setting them equal yields

$$x^\top \Sigma^{-1} \mu_1 - \frac{1}{2} \mu_1^\top \Sigma^{-1} \mu_1 + \log(\pi_1) = x^\top \Sigma^{-1} \mu_0 - \frac{1}{2} \mu_0^\top \Sigma^{-1} \mu_0 + \log(\pi_0) \quad (3)$$

$$x^\top \Sigma^{-1} (\mu_1 - \mu_0) - \frac{1}{2} (\mu_1^\top \Sigma^{-1} \mu_1 - \mu_0^\top \Sigma^{-1} \mu_0) + \log\left(\frac{\pi_1}{\pi_0}\right) = 0 \quad (4)$$

This is a linear function of x , confirming that the LDA decision boundary is a hyperplane in $\mathbb{R}^p$. To interpret this in terms of the Mahalanobis distance, recall that the squared Mahalanobis distance from a point x to a distribution of p_{xy} conditioned on class g is

$$d_{\mathcal{M}}^2(x, \mu_g) = (x - \mu_g)^\top \Sigma^{-1} (x - \mu_g) \quad (5)$$

Expanding both $d_{\mathcal{M}}^2(x, \mu_1)$ and $d_{\mathcal{M}}^2(x, \mu_0)$ and taking their difference

$$\begin{aligned} d_{\mathcal{M}}^2(x, \mu_1) - d_{\mathcal{M}}^2(x, \mu_0) &= (x - \mu_1)^\top \Sigma^{-1} (x - \mu_1) - (x - \mu_0)^\top \Sigma^{-1} (x - \mu_0) \\ &= -2x^\top \Sigma^{-1} (\mu_1 - \mu_0) + \mu_1^\top \Sigma^{-1} \mu_1 - \mu_0^\top \Sigma^{-1} \mu_0 \end{aligned} \quad (6)$$

Substituting back to (4) multiplied by 2, the decision boundary can be equivalently written as:

$$d_{\mathcal{M}}^2(x, \mu_1) - d_{\mathcal{M}}^2(x, \mu_0) = 2 \log\left(\frac{\pi_1}{\pi_0}\right) \quad (7)$$

Therefore, from $\delta_1(x) - \delta_0(x) > 0$ we deduce that LDA classifies x to class 1 iff

$$d_{\mathcal{M}}^2(x, \mu_0) - d_{\mathcal{M}}^2(x, \mu_1) > 2 \log\left(\frac{\pi_1}{\pi_0}\right) \quad (8)$$

That is, LDA assigns x to the class whose mean μ_g is closest in Mahalanobis distance adjusted by the log prior ratio $\log(\pi_1/\pi_0)$. When the classes are equally likely, the boundary reduces to the pure Mahalanobis midpoint $d_{\mathcal{M}}^2(x, \mu_1) =$

$d_{\mathcal{M}}^2(x, \mu_0)$, i.e. the set of points equidistant from both class means under the metric induced by assuming the equal inverse covariance matrix Σ^{-1} .

2. Now, relax the equal covariance assumption. Derive the decision boundary for Quadratic Discriminant Analysis (QDA). Explain, from a geometric perspective, how this boundary differs from LDA's and under what data conditions QDA is expected to perform significantly better.

Relaxing the equal covariance assumption, we now allow each class $g \in \{0, 1\}$ to have its own covariance matrix Σ_g . The class-conditional density is:

$$p(X | Y = g) = \frac{1}{(2\pi)^{p/2} |\Sigma_g|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_g)^\top \Sigma_g^{-1} (x - \mu_g) \right) \quad (9)$$

The log posterior for class g yields the quadratic discriminant function

$$\delta_g(x) = -\frac{1}{2} \log |\Sigma_g| - \frac{1}{2} (x - \mu_g)^\top \Sigma_g^{-1} (x - \mu_g) + \log(\pi_g) \quad (10)$$

The decision boundary is again defined by $\delta_1(x) = \delta_0(x)$, which yields

$$-\frac{1}{2} \log |\Sigma_1| - \frac{1}{2} (x - \mu_1)^\top \Sigma_1^{-1} (x - \mu_1) + \log(\pi_1) = -\frac{1}{2} \log |\Sigma_0| - \frac{1}{2} (x - \mu_0)^\top \Sigma_0^{-1} (x - \mu_0) + \log(\pi_0) \quad (11)$$

$$\frac{1}{2} (x - \mu_0)^\top \Sigma_0^{-1} (x - \mu_0) - \frac{1}{2} (x - \mu_1)^\top \Sigma_1^{-1} (x - \mu_1) = \frac{1}{2} \log \frac{|\Sigma_0|}{|\Sigma_1|} + \log \left(\frac{\pi_1}{\pi_0} \right) \quad (12)$$

In terms of Mahalanobis distances, the QDA decision boundary is written as

$$d_{\mathcal{M}}^2(x, \mu_0, \Sigma_0) - d_{\mathcal{M}}^2(x, \mu_1, \Sigma_1) = \log \frac{|\Sigma_0|}{|\Sigma_1|} + 2 \log \left(\frac{\pi_1}{\pi_0} \right) \quad (13)$$

where $d_{\mathcal{M}}^2(x, \mu_g, \Sigma_g) = (x - \mu_g)^\top \Sigma_g^{-1} (x - \mu_g)$ is now class-specific. Unlike LDA where Σ cancels and yields a linear boundary, here the boundary contains quadratic terms $x^\top \Sigma_g^{-1} x$ from each class, making the boundary a quadratic surface.

Geometric interpretation. LDA produces a linear decision boundary, i.e., a hyperplane that partitions $\mathbb{R}^p$ using the difference in class means and a shared covariance structure. In contrast, QDA yields a quadratic decision boundary because each class has its own covariance matrix Σ_g , generating a more complex separating surface. QDA tends to outperform LDA when the true class covariances differ, because when you can make less assumption in mathematics there always yield more powerful result in the presence of the data set. In a technical setting, QDA perform better than LDA when the sample size is large relative to the dimensionality ($\kappa = n/p \gg 1$) so that $\hat{\Sigma}_0$ and $\hat{\Sigma}_1$ is non-singular matrix. Lastly if we are able to plot the data points and see the nonlinear boundary with our eyes and hypothesis a quadratic boundary. However as noted from our discussion that hypothesizing the function space in non-trivial dimensions directly from the dataset $\mathcal{D}_n$ with a robust insight on $\mathcal{H}(\mathcal{D}_n)$ is still under the works in categorical theory.

B. Data Analysis & High-Dimensional Diagnostics

We analyze the prostate dataset (prostate-cancer-1.csv), a DNA microarray gene expression dataset.

1. Load the data. Report n , p , and the ratio $\kappa = n/p$. Discuss the implications of κ being much less than 1 for covariance matrix estimation and classifier stability.

We have the dimension of the data matrix $\dim(X) = 79 \times 500$ (i.e. $n = 79, p = 500$) with a $\kappa = n/p = 0.158$. Clearly, this is an under-determined system where we have less samples than predictor variables. Recall that the sample covariance matrix is estimated as

$$\hat{S} = \frac{1}{n-1} X^T X \quad (14)$$

where X is the $n \times p$ centered data matrix. Since $n < p$, the matrix $X^T X$ is a $p \times p$ matrix with rank at most $\min(n, p) = n = 79$. This means $\hat{\Sigma}$ is rank-deficient with $\text{rank} \leq 79 < 500$, making it singular.

2. Estimate the class prior $\hat{\pi}$ and compute $\hat{\eta} = (1 - \hat{\pi})/\hat{\pi}$. Provide a detailed interpretation of $\hat{\eta}$ in the context of class imbalance and its effect on classifier decision thresholds.

We have a binary class $y = \{0, 1\}$ from this data. If we define $\hat{\pi}$ as the proportion of samples in class 1, namely $y = 1$

$$\hat{\pi}_1 = \frac{1}{n} \sum_{1 \leq i \leq n} \mathbb{1}\{y_i = 1\} = \frac{n_1}{n} = 0.5316456 \quad (15)$$

similarly, if we let $\hat{\pi} = \hat{\pi}_0$ then

$$\hat{\pi}_0 = \frac{1}{n} \sum_{1 \leq i \leq n} \mathbb{1}\{y_i = 0\} = \frac{n_0}{n} = 0.4683544 \quad (16)$$

Since this is a binary class then we must have $\hat{\pi}_0 + \hat{\pi}_1 = 1$, then $\hat{\eta}_0 = \hat{\pi}_1/\hat{\pi}_0 = 1.135135$ and $\hat{\eta}_1 = \hat{\pi}_0/\hat{\pi}_1 = 0.8809524$. So intuitively, $\hat{\eta}$ of either class it just saying which class is has more samples based on this prior. For $\hat{\eta}_0$ we saying there is 1.135135 more samples in class 1 than class 0. This $\hat{\eta}$ is extremely important because in the presence of class imbalance, classifiers may need adjusted decision thresholds to account for the prior probabilities and avoid bias toward the majority class. In this case I would say the two classes are approximately balanced and there should be too much issues.



Figure 1: $n_0 = 37, n_1 = 42$

3. Estimate the class-specific mean vectors $\hat{\mu}_0, \hat{\mu}_1$ and covariance matrices $\hat{\Sigma}_0, \hat{\Sigma}_1$.

Let $n = n_0 + n_1$ then we have the mean vector and covariance class specific estimates be $\hat{\mu}_k \in \mathbb{R}^{500}, \hat{\Sigma}_k \in \mathbb{R}^{500 \times 500}$

$$\hat{\mu}_0 = \frac{1}{37} \sum_{1 \leq i \leq 79} \vec{x}_i \mathbb{1}\{y_i = 0\}, \quad \hat{\mu}_1 = \frac{1}{42} \sum_{1 \leq i \leq 79} \vec{x}_i \mathbb{1}\{y_i = 1\} \quad (17)$$

$$\hat{\Sigma}_0 = \frac{1}{36} \sum_{i=1}^{79} (\vec{x}_i - \hat{\mu}_0)(\vec{x}_i - \hat{\mu}_0)^T \mathbb{1}\{y_i = 0\}, \quad \hat{\Sigma}_1 = \frac{1}{41} \sum_{i=1}^{79} (\vec{x}_i - \hat{\mu}_1)(\vec{x}_i - \hat{\mu}_1)^T \mathbb{1}\{y_i = 1\} \quad (18)$$

- Compute $\|\hat{\mu}_1 - \hat{\mu}_0\|_2$. What does this suggest about the separability of the classes?

$$\|\hat{\mu}_1 - \hat{\mu}_0\|_2 = \left[\sum_{j=1}^{500} (\hat{\mu}_{1j} - \hat{\mu}_{0j})^2 \right]^{1/2} = 2.102082 \quad (19)$$

From a L2 distance of 2.102082, we can at most confirm that these two class exist in some form of unique space where they do not overlap in $\mathbb{R}^{500}$, however we do not have enough information to make a definitive statement about the separability of the class without knowing the within group distances and possible overlapping data points.

- Compute the Frobenius norm $\|\hat{\Sigma}_1 - \hat{\Sigma}_0\|_F$. Discuss the validity of LDA's equal covariance assumption for this data.

We compute the Frobenius norm of the difference between two covariance matrix as

$$\|\hat{\Sigma}_1 - \hat{\Sigma}_0\|_F = \left[\sum_{i=1}^{500} \sum_{j=1}^{500} (\hat{\Sigma}_{1,ij} - \hat{\Sigma}_{0,ij})^2 \right]^{1/2} = 3.713011 \quad (20)$$

A clear result from this can be said is that the Frobenius norm of the difference between covariance is much larger than 0, indicating that assumption of equal variance $\hat{\Sigma}_0 = \hat{\Sigma}_1$ for LDA might not be valid. However since the covariance matrix is rank deficit as mention from computing $\kappa \ll 1$, they may be huge estimator error in high dimensions. So we are advocating for the hope of LDA still being "effective".

$$\hat{\Sigma}_1 - \hat{\Sigma}_0 = \underbrace{(\Sigma_1 - \Sigma_0)}_{\text{True Variance}} + \underbrace{(\hat{\Sigma}_1 - \Sigma_1) - (\hat{\Sigma}_0 - \Sigma_0)}_{\text{Estimation Error}} \quad (21)$$

4. Covariance Eigen-Analysis:

- Compute the pooled covariance estimate $\hat{\Sigma}$ and the overall sample covariance $\hat{S}$. Plot both matrices and comment on their similarities/differences.

We compute the Sample Covariance and Pooled Covariance with $G = 2$ as

$$\hat{S} = \frac{1}{n-1} X^T X, \quad \hat{\Sigma}_{pooled} = \frac{\sum_{1 \leq g \leq G} (n_g - 1) \hat{S}_g}{\sum_{1 \leq g \leq G} (n_g - 1)} \quad (22)$$

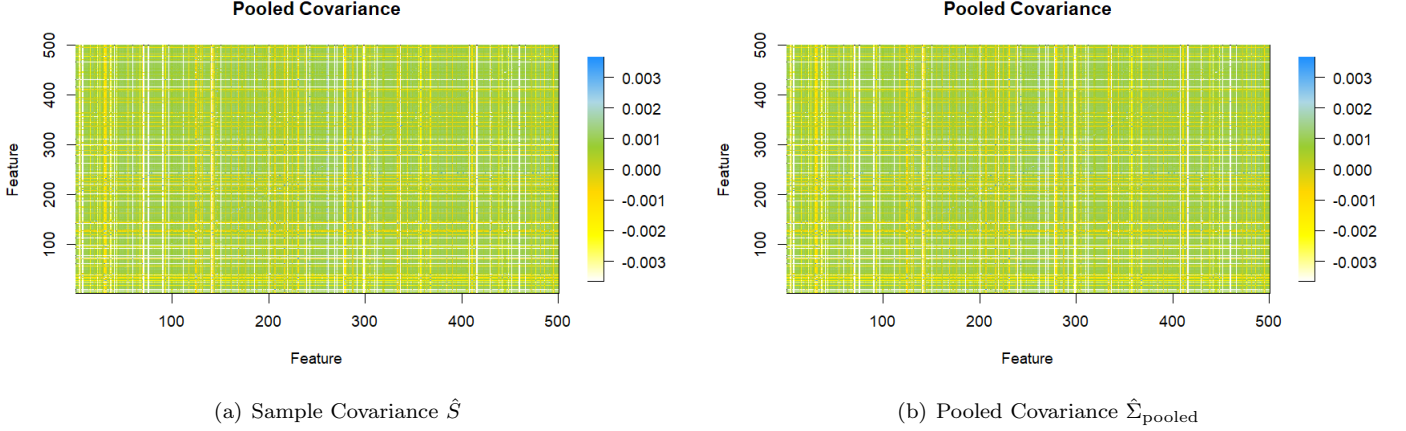


Figure 2: Comparison of Sample and Pooled Covariance Matrices

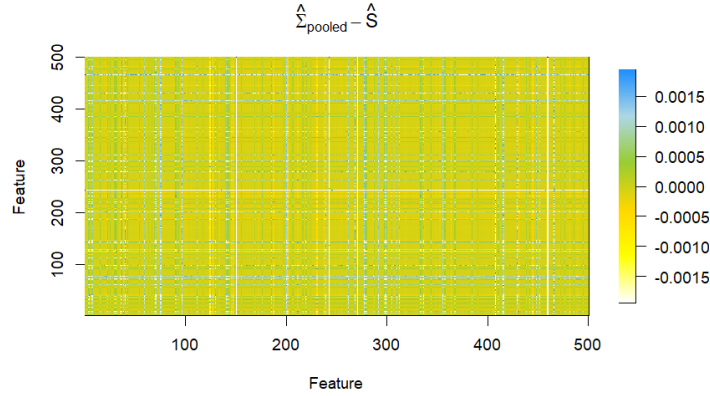


Figure 3: Difference in Covariance

Based on the difference in covariance figure, we can see that the difference between the pooled covariance matrix and the sample covariance matrix is nearly 0. The Frobenius norm $\|\hat{\Sigma}_1 - \hat{\Sigma}_0\|_F = 3.713011$ is notably large relative to zero, suggesting the two class-specific covariance matrices differ. However, we mentioned that this result must be interpreted cautiously in the high-dimensional setting where $\kappa = 0.158 \ll 1$. Since both $\hat{\Sigma}_0$ and $\hat{\Sigma}_1$ are rank-deficient. Based on $\hat{\Sigma}_{pooled} \approx \hat{S}$, as shown in Figure 2, Since the classes are approximately balanced, with the pooled estimator converges to the overall sample covariance, suggesting the within-class covariance structure is not dramatically different between classes. Therefore, LDA's equal covariance assumption $\Sigma_0 = \Sigma_1$ is not strongly violated by the data, and LDA remains loosely reasonable for this classification task.

- Perform an eigen-decomposition of $\hat{\Sigma}$. Plot the scree plot (first 20 eigenvalues). Describe the intrinsic dimensionality of the data and the implications for dimensionality reduction before classification.

Since $\hat{\Sigma}_{pooled}$ is a square matrix with linearly independent eigenvectors. Then we can perform eigen-decomposition

$$\hat{\Sigma}_{pooled} = Q \Lambda Q^{-1} \quad (23)$$

where Q is a square matrix with column as eigenvectors and Λ be the diagonal matrix with eigenvalues λ_i corresponding to the eigenvectors $\vec{q}_i$ in Q .

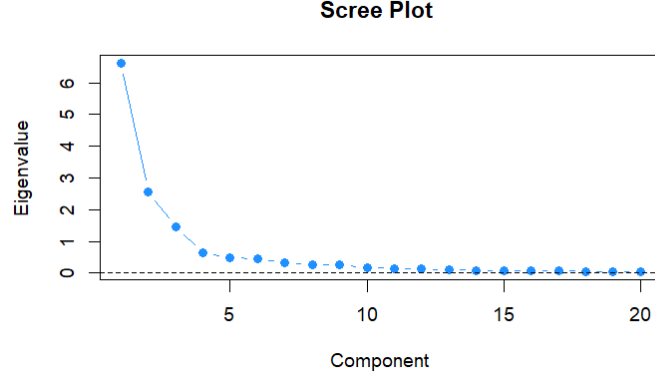


Figure 4: Scree Plot

Plotting the first 20 eigenvalues on the scree plot allows us to notice that we can reduce own dimension to about 11 and still maintain similar variability and information about the pooled sampled covariance. We choose to keep 11 components because after the 11 component the eigenvalues are near 0 which in a way means that it has little to no effects in scaling the sample covariance matrix. In summary we can possibly reduce the feature space in $\mathbb{R}^{500}$ to be projecting into a subspace in 11 dimensions where the components are some linear combinations of the original features. We can maintain the essential variation in the data perform PCA, LDA or QDA after such projection.

$$\hat{\Sigma}_{pooled} \vec{q}_i = \lambda_i \vec{q}_i, \quad \vec{q}_i = \sum_{1 \leq j \leq p} d_j \vec{e}_j \quad (24)$$

where $\vec{e}_j$ is the j th basis in the original feature space $\mathbb{R}^{500}$.

5. Test the assumption of multivariate normality for each class-conditional distribution using tests from `mvnrmtest` or `MVN`. Report your findings and their consequences for LDA/QDA.

Using the `mvnrmtest` with the `mshapiro.test` function, we compute the p-values of the W^* statistics and plot the QQ-plots of the 6 random QQ-plots of each features from each class and see that the class conditional density for some feature $j \in [1, 500]$ is approximately normal, but the multivariate joint conditional density given the class is not based on the multivariate Shapiro Wilk test.

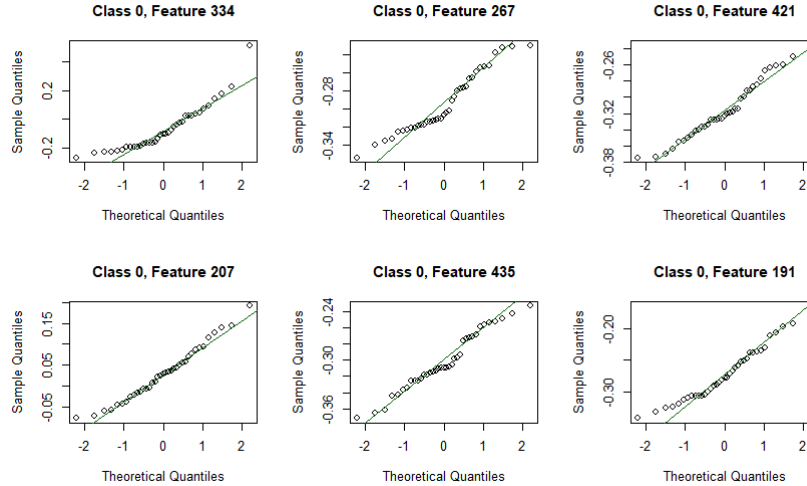


Figure 5: QQ Plot of features $x_j|y = 0$

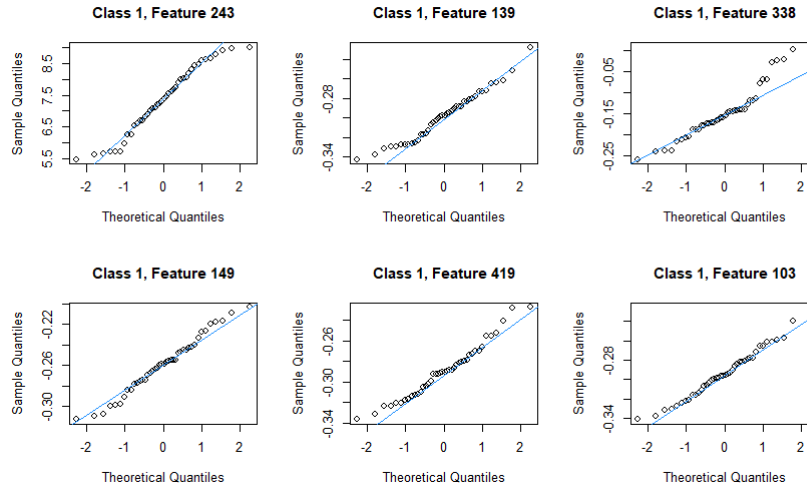


Figure 6: QQ Plot of features $x_j|y = 1$

A random sample $\vec{x}_1, \dots, \vec{x}_n \sim N_P(\vec{\mu}^*, \Sigma^*)$ if W^* computed with a test size $\gamma \in (0, 1)$ for some critical value $c_{\gamma;n,d}$ approximate by normal quantiles is

$$W^* = \left(\frac{1}{p} \sum_{1 \leq j \leq p} w_{\hat{z}_j} \right) < c_{\gamma;n,d} \quad (25)$$

To honestly report, the multivariate Shapiro-Wilk test rejects normality for both classes at the $\alpha = 0.05$ level with a p-value of 1.439938×10^{-12} for class 0 and 3.846867×10^{-12} for class 1. However, we can argue that given the small sample sizes ($n_0 = 37, n_1 = 42$) and the high-dimensional setting ($p = 500$), this result should be interpreted with caution. The univariate Q-Q plots suggest approximate marginal normality for individual features, and Naive Bayes is known to be robust to moderate violations of multivariate normality, particularly when class sizes are small. Therefore, we proceed with Naive Bayes and LDA as a reasonable classifier while acknowledging this limitation.

6. Create a 2×3 panel of comparative boxplots for the first 6 features, colored by class. Identify any features showing promising marginal separation.

Plotting the first 6 features colored by class yields a not so apparent separation.

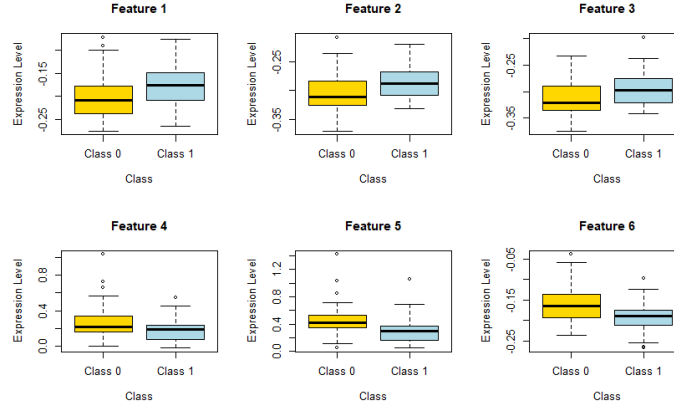


Figure 7: Box Plots for features $j \in [0, 6]_{\mathbb{Z}}$

Then we suppose that there might be other features in the 500 features that may show more distinct separation between class, as we feel that picking the first 6 features is biased. Then we perform a t -test for two sample means between the two classes and choose the 6 best features that have the largest difference in means with the hypothesis test for $j \in [1, 500]_{\mathbb{Z}}$

$$\begin{aligned} H_0 : (\mathbb{E}(x_j)|y=0) &= (\mathbb{E}(x_j)|y=1) \\ H_a : (\mathbb{E}(x_j)|y=0) &\neq (\mathbb{E}(x_j)|y=1) \end{aligned} \quad (26)$$

using the t-statistics without assuming equal variance

$$t_j = \frac{\hat{\mathbb{E}}(x)_{j,1} - \hat{\mathbb{E}}(x)_{j,0}}{\text{SE}} = \frac{\hat{\mathbb{E}}(x)_{j,1} - \hat{\mathbb{E}}(x)_{j,0}}{\sqrt{\frac{\hat{s}_{j,0}^2}{n_0} + \frac{\hat{s}_{j,1}^2}{n_1}}} \quad (27)$$

with $\hat{s}_{j,g}^2 = \sum_{i|y_i=g} (x_{ij} - \mathbb{E}(x_{j,g}))^2 / (n_g - 1)$. The idea is that we take the features with the largest $|t|$ value in $[1, 500]$, in other words with the 6 smallest p-value that rejects H_0 , then we say those 6 are the top features with the greatest difference in means. This process yields the top features as features 125, 445, 278, 58, 40 and 446.

$$\tilde{t}_j = \arg \max_{j \in \{0, 500\}} |t_j| \quad (28)$$

Yet notice that the features alone still does not show any significant separation between the two classes.

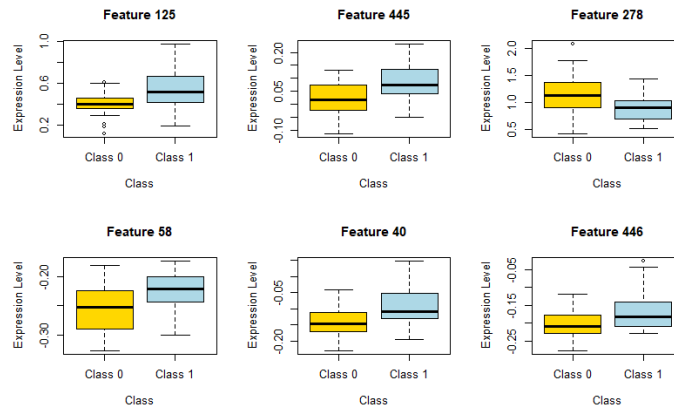


Figure 8: Top 6 features via two sample t-test

7. Based on all your diagnostics, which method—LDA, QDA, Naive Bayes, or Logistic Regression—would you a priori expect to perform best on this data? Justify your reasoning thoroughly, linking it to the data properties you have uncovered.

Based on the diagnostics, we expect **LDA** to perform best a priori. Since $\kappa = 0.158 \ll 1$ with $n = 79$ and $p = 500$, the system is severely underdetermined, making QDA immediately infeasible as it requires estimating two full 500×500 covariance matrices each with rank ≤ 79 .

Naive Bayes assumes feature independence which is unrealistic for gene expression data where genes are known to be co-expressed in biology which makes the Naive Bayes perform poorly in comparison to LDA. Logistic Regression is similarly unstable when $p \gg n$ without regularization. In contrast, LDA only requires estimating a single pooled covariance matrix, and our diagnostics support this choice: $\hat{\Sigma}_{\text{pooled}} \approx \hat{S}$ and the Frobenius norm $\|\hat{\Sigma}_1 - \hat{\Sigma}_0\|_F = 3.713$ is likely dominated by estimation error rather than a true structural difference between classes. Furthermore, the approximately balanced classes ensure the prior $\hat{\eta}$ does not severely bias the decision threshold, and LDA's implicit dimensionality reduction via the pooled covariance makes it the most stable classifier in this high-dimensional setting. Note that QDA and Logistic Classifier failed to run for these data sets.

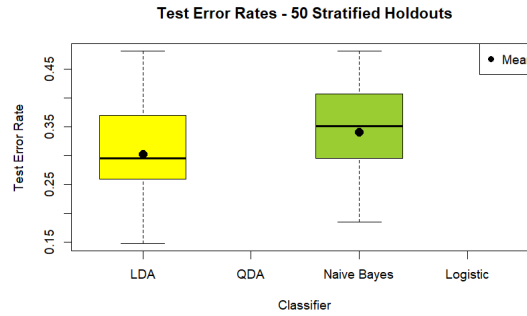


Figure 9: Classifier Performance Boxplot

Exercise 2: Extrinsic Empirical Comparisons (30 points)

Use the spam dataset from the kernlab package for a comparative study.

1. Describe the dataset: sample size, dimensionality, and the nature of the features (e.g., continuous percentages, counts, etc.). Plot the class distribution.

In the spam dataset we have a binary class with classification task with a data matrix $\mathbf{X} \in \mathbb{R}^{4601 \times 57}$. There is also $n = 4601$ binary responses of $\mathbf{Y} \in \{\text{Non-spam}, \text{Spam}\}$. From the dataset description from `help("spam")` we have that

The first 48 variables contain the frequency of the variable name (e.g., business) in the e-mail. If the variable name starts with num (e.g., num650) then it indicates the frequency of the corresponding number (e.g., 650). The variables 49-54 indicate the frequency of the characters ';', '(', '[', '!', '\$', and '#'. The variables 55-57 contain the average, longest and total run-length of capital letters. Variable 58 indicates the type of the mail and is either "nonspam" or "spam", i.e. unsolicited commercial e-mail.

To summarize,

1. **Word frequency variables (variables 1–48):** The percentage of words in the email that match specific words (e.g., "business", "credit", "george"). These are continuous variables bounded between 0 and 100, representing the relative frequency of each word.
2. **Character frequency variables (variables 49–54):** The percentage of characters in the email that are special symbols, specifically ';', '(', '[', '!', '\$', and '#'. Like the word frequencies, these are continuous variables bounded between 0 and 100.
3. **Capital letter statistics (variables 55–57):** Three measures of capital letter usage:
 - `capital_run_length_average`: Average length of uninterrupted sequences of capital letters
 - `capital_run_length_longest`: Length of the longest uninterrupted capital letter sequence
 - `capital_run_length_total`: Total number of capital letters in the email

The class distribution of 2788 samples with response as non-spam and 1813 samples with response as spam.

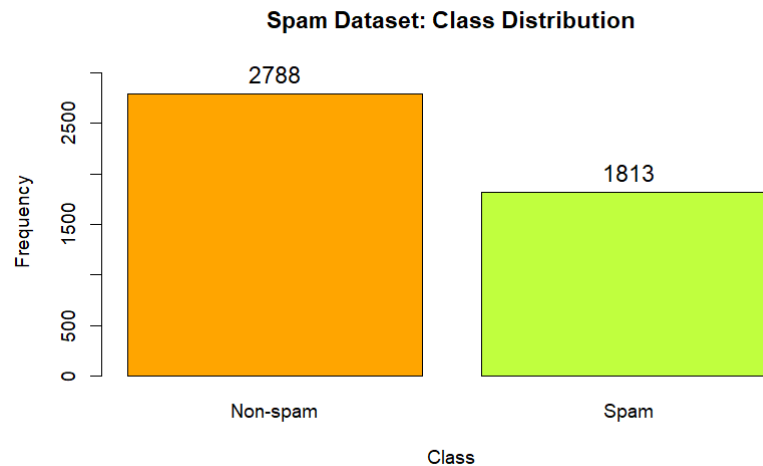


Figure 10: Class Distribution

2. Implement a performance evaluation pipeline to compare LDA, QDA, Naive Bayes, 9NN, and SVM (with a linear kernel).

- First, evaluate them on the entire dataset (resubstitution method) and plot all five ROC curves on the same axes. Comment on the performance hierarchy and its plausibility.

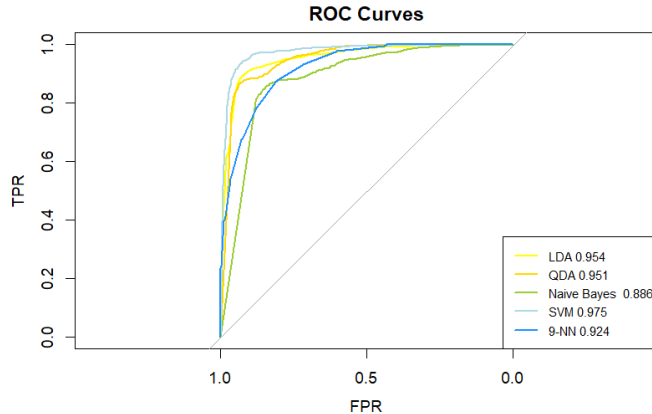


Figure 11: ROC Curves & AUC Values

The computed AUC values for each classifier in our consideration is for LDA = 0.9543, QDA = 0.9509, Naive Bayes = 0.8862, SVM = 0.9752, 9-NN = 0.9241. In our understanding, we can interpret the AUC as the probability under each individual ROC curve which is bounded between 0 and 1. Namely if we consider M to be the set of classifiers under consideration,

$$\text{AUC}(m \in M) = \Pr(\hat{p}_m(1) > \hat{p}_m(0) | \hat{f}_m(x) = y) \quad (29)$$

where $\hat{p}$ in this case the probability assigned from the statistical classifier in set M . This is equivalent to saying that the AUC is the chance that the probabilities calculated by the binary classifier will be higher for the positive class than the negative class when given the positive classification is actually positive and the negative classification is also negative. The ROC curve is realistically a step function as the sample sample size n will be finite, with at most $n + 1$ unique pair of TPR and FPR from recalculating the threshold, assuming there is no ties in probabilities. A proof can be provided in some way if we represent the TPR as a function of the FPR and integrate accordingly.

- Second, using `set.seed(7472025)`, perform 50 repeated stratified holdouts (2/3 train, 1/3 test). For each replication, compute the test error rate for each classifier.

We attempted 50 repeated stratified holdout experiments (2/3 training, 1/3 test). Due to numerical failures in the QDA classifier during model fitting, only 27 runs completed successfully. The results reported below are therefore based on the 27 valid replications for QDA. We can say with confidence SVM out performs all other classifiers for the Spam dataset.

Classifier	Mean Test Error	Std. Dev.
LDA	0.113	0.008
QDA	0.169	0.009
Naive Bayes	0.292	0.010
9NN	0.218	0.008
SVM	0.072	0.006

Table 1: Unadjusted Summary of 50 Stratified Holdout Test with Rounded Errors

3. Create comparative boxplots of the 50 test errors for each method.

- Analyze the central tendency and variance of the test errors. Which method appears most stable?

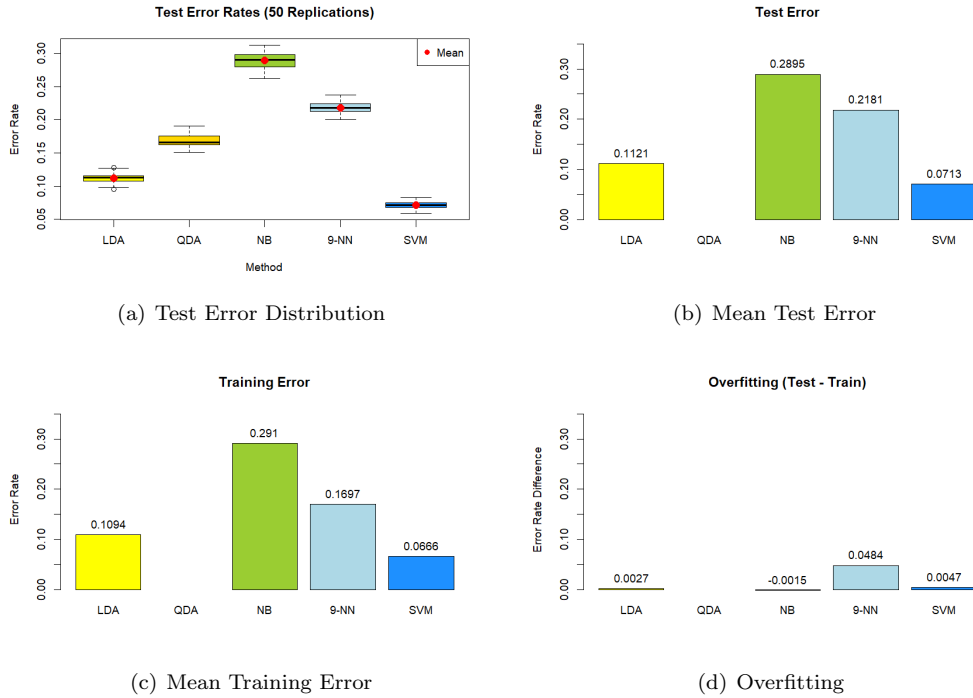
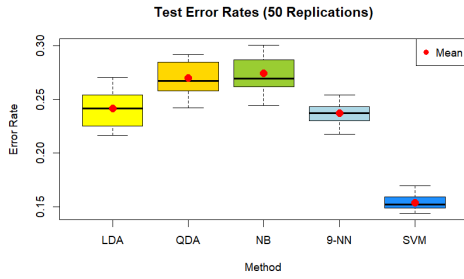


Figure 12: Performance comparison of five classifiers on spam data

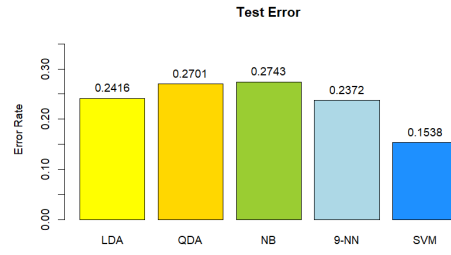
The second idea for dealing with the QDA not being able to perform is that we can remove the predictor variables that are near 0, therefore making our data matrix invertible again. Using the `nearZeroVar()` function in the library(`caret`) we remove 50 predictor variables and leaving 8 that allows QDA to run all 50 stratified holdouts. To be fair to the other statistical classifiers, we run them with the same number of predictor variables. Essentially the new data matrix is $X_{new} \in \mathbb{R}^{4601 \times 7}$. This yields the adjusted results will worse performance overall as we now have less information for the machines to learn from. Yet again we can say that SVM outperforms all other four models and is the most stable once again. While QDA isn't really suited for a task like as much adjustments were needed, and we may have just relied on other classifiers for this task for better results.

Classifier	Mean Test Error	Std. Dev.
LDA	0.2415	0.0152
QDA	0.2701	0.0149
NB	0.2743	0.0156
9-NN	0.2372	0.0092
SVM	0.1538	0.0063

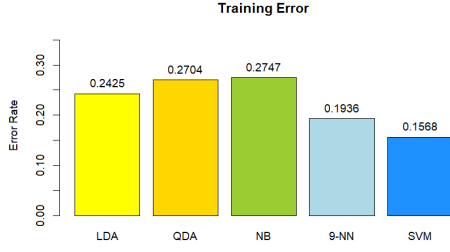
Table 2: Adjusted Summary of 50 Stratified Holdout Test with Rounded Errors



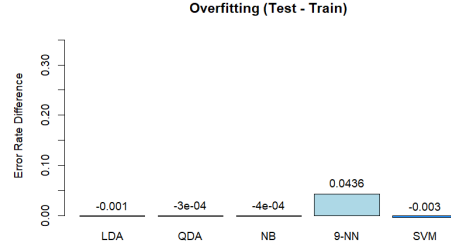
(a) *Adjusted Test Error Distribution*



(b) *Adjusted Mean Test Error*



(c) *Adjusted Mean Training Error*



(d) *Adjusted Overfitting*

Figure 13: Performance comparison of five classifiers on spam data after removing 50 near-zero variance predictors

- Discuss the results in the context of the bias-variance trade-off and the complexity of each model, especially given the high-dimensionality of the feature space.

The results clearly illustrate the bias-variance trade-off among the five classifiers. **SVM with linear kernel** achieves the lowest test error (0.072 on full data, 0.154 on reduced data) with minimal variance, striking an optimal balance between bias and variance for this high-dimensional problem. **LDA** makes a strong parametric assumption (equal covariances) which introduces some bias but maintains low variance, performing well with 57 features. **QDA** relaxes the equal covariance assumption, increasing model complexity and variance, which explains its higher error and frequent numerical failures when $p > n$ it simply cannot estimate class-specific covariance matrices reliably with 57 features and limited samples. **9-NN** is inherently high-variance due to its local decision boundary, and this is further pushed by the curse of dimensionality in 57-dimensional space, causing its error rate to rise substantially. **Naive Bayes** makes the extremely strong (and clearly false) assumption of conditional independence among features, introducing substantial bias that dominates any variance reduction, resulting in the poorest performance. After removing near-zero variance predictors (reducing to 8 features), all methods show increased bias due to information loss, but the relative ordering remains consistent, confirming that SVM's linear decision boundary is best suited for this spam classification task.

Exercise 3: Theoretical Foundations & Proofs (20 points)

Prove the following fundamental results in statistical learning and data mining.

1. k-Nearest Neighbors (kNN): Show that the kNN classification rule is equivalent to a locally weighted majority vote. Under the condition that $k \rightarrow \infty$ and $k/n \rightarrow 0$, argue heuristically why kNN can be a consistent classifier. (4 points)

Let $C(\cdot)$ be the true label of some data point in $\mathcal{D}_n$. From a Bayesian prospective we can view the classification problem of interest be the posterior density for a new observation point x_0

$$P(C(x_0) = g | X = x_0) = \frac{P(X = x_0 | C(x_0) = g)P(C(x_0) = g)}{P(X = x_0)} \quad (30)$$

The prior of x_0 being class g is best written as

$$\hat{P}(C(x_0) = g) = \frac{|\mathcal{D}_g|}{|\mathcal{D}_n|} = \frac{n_g}{n} \quad (31)$$

Let $S(k_g) \subsetneq S(k)$ denote the subset of nearest k_g neighbors to x_0 that is also in class g and let $V_k(x_0)$ be the volume of cell or closed ball around the k nearest points to x_0 then

$$\hat{P}(X = x_0 | C(x_0) = g) = \frac{k_g}{n_g V_k(x_0)} \quad (32)$$

$$\hat{P}(X = x_0) = \frac{k}{n V_k(x_0)} \quad (33)$$

By Bayes Theorem we can obtain that posterior density under the conditions x_0 for $k \rightarrow \infty$ and $k/n \rightarrow 0$ so we can maintain a consistent volume $V_k(x_0)$ as n grows.

$$\hat{P}(C(x_0) = g | X = x_0) = \frac{\frac{k_g}{n_g V_k(x_0)} \frac{n_g}{n}}{\frac{k}{n V_k(x_0)}} = \frac{k_g}{k} \quad (34)$$

This formulation “pays” well as we can clearly see that under the conditions above we have a consistent weighted probability for classifying a point in group g based on the size of $S(k_g)$ in $S(k)$. This is a very intuitive proportion as the probability of $C(x_0) = g$ given that $X = x_0$ under kNN is the number of points in $S(k)$ in class g divided by the total number of k neighbors.

2. Principal Component Analysis (PCA): Let X be an $n \times p$ data matrix with $n \ll p$. Show that performing PCA on the $p \times p$ covariance matrix $S = \frac{1}{n-1} X^\top X$ is computationally inefficient. Prove that the eigenvectors of $X^\top X$ can be obtained from the eigenvectors of the $n \times n$ Gram matrix $XX^\top$. (6 points)

Note that the covariance matrix $S \in \mathbb{R}^{p \times p}$ is highly expensive to compute since $p \gg n$, then the complexity is already $\mathcal{O}(np^2)$, since there is p^2 multiplications for each inner products to compute each of length n . Then from here we have to perform eigen decomposition on S ,

$$S\vec{v}_i = \lambda_i \vec{v}_i, \quad i = 1, \dots, p \quad (35)$$

In general solving a this set of equations for a $p \times p$ matrix will be at a cost of $\mathcal{O}(p^3)$. To mention again that $p \gg n$ so perform these two task is not a efficient. Interestingly, if we look at the rank of S

$$\text{rank}(S) = \text{rank}(X^\top X) = \text{rank}(X) = \min(n, p) = n \quad (36)$$

Proof. Recall the rank nullity theorem from linear algebra

$$\text{rank}(A) = p - \text{nullity}(A) \quad (37)$$

This motivates us to show $\dim \text{null}(X^\top X) = \dim \text{null}(X)$ with double inclusion of the two null spaces.

$$\begin{aligned} X\vec{u} = \vec{0} \rightarrow X^\top \vec{0} = \vec{0} &\Rightarrow \text{null}(X) \subset \text{null}(X^\top X) \\ X^\top X\vec{u} = \vec{0} \rightarrow \vec{u}^\top X^\top X\vec{u} = \vec{0} \rightarrow \|X\vec{u}\|^2 = 0 &\iff X\vec{u} = \vec{0} \Rightarrow \text{null}(X^\top X) \subset \text{null}(X) \end{aligned} \quad (38)$$

This means that this process takes high computation complexity and we only in return get at most n nonzero valued $\lambda_i > 0$, and this is important in PCA as λ_i represent the amount of variance in the original data captured by each corresponding principal component. $\square$

Note that the Gram matrix $XX^T \in \mathbb{R}^{n \times n}$ which will be significantly more efficient than performing *PCA* on S . We show that every nonzero eigenvalue of $X^T X$ is an eigenvalue of XX^T , and vice versa.

Proof. Let $\lambda \neq 0$ be an eigenvalue of $X^T X$ with eigenvector $v \neq 0$

$$X^T X v = \lambda v. \quad (39)$$

Multiplying on the left by X

$$XX^T(Xv) = \lambda(Xv). \quad (40)$$

So $u = Xv$ is an eigenvector of XX^T with eigenvalue λ , provided $u \neq 0$. Since $\lambda \neq 0$, if $Xv = 0$ then $X^T X v = 0$, which contradicts $\lambda v \neq 0$. So $u \neq 0$. The argument runs identically in reverse. Let $\lambda \neq 0$ be an eigenvalue of XX^T with eigenvector $u \neq 0$

$$XX^T u = \lambda u. \quad (41)$$

Multiplying on the left by X^T

$$X^T X(X^T u) = \lambda(X^T u). \quad (42)$$

So $v = X^T u$ is an eigenvector of $X^T X$ with eigenvalue λ , and by the same argument $v \neq 0$ since $\lambda \neq 0$. This result is a special case of the lemma from Jacobson's lemma [1]. □

3. Support Vector Machines (SVM): Derive the dual optimization problem for the linear SVM with a soft margin (hinge loss). Clearly state the constraints and explain the interpretation of the Lagrange multipliers α_i . (5 points)

The dual problem arises from supervised learning, specifically in binary classification support vector machines. We have the data and response in the form $(x_i, y_i), x_i \in \mathbb{R}^p, y_i \in \{-1, +1\}$ for $i = 1, 2, \dots, n$. Then suppose that there exist a hyperplane that can linearly separate the data into the two classes with two margins that is equally distanced from the hyperplane. Our goal is to maximize such margin so that we can have the clearest decision boundary in either the feature space or latent space $\mathcal{F}$. Also to mention that the margins are in the form of $w^T x_i + b = \pm 1$, where we define the positive class of x_i to be on the $w^T x_i + b = +1$ or above, and the classified negative x_i to be on the margin $w^T x_i + b = -1$ and below. More compactly we can write the decision bound with the signum function

$$\hat{f}(x_i) = \text{sgn}(w^T x_i + b) \quad (43)$$

Explicitly, if we define x_j to be a point on the negative margin then $w^T x_j + b = -1$ and the closest possible point x_k if there exist one to x_j on the positive class must lie on the positive margin, say $w^T x_k + b = +1$. Then the point has the relation $x_k = \beta w + x_j$ because w is the normal vector to both planes, as the planes was written in the normal form. It follows that we make the substitution

$$\begin{aligned} w^T(\beta w + x_j) + b &= +1 \\ w^T \beta w + w^T x_j + b &= +1 \\ \beta w^T w &= 2 \\ \beta &= \frac{2}{\|w\|^2} \end{aligned} \quad (44)$$

In this context we have β be the shortest distance between two parallel margins, circling back to our goal, we want to maximize this distance, which is equivalent to minimizing $\|w\|^2/2$ with subjective to $y_i(w^T x_i + b) \geq 1$. The problem also arises with this constraint when the data is not linearly separable in $\mathcal{F}$, so we add an additional constraint so it eases the condition and $y_i(w^T x_i + b) \geq 1 - \xi_i$ as this requires the hyperplane to correctly classifier all $x_i \forall i$. Introduce the soft margin primal problem as

$$\min_{w, b, \xi_i} \frac{\|w\|^2}{2} + C \sum_{1 \leq i \leq n} \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad 1 \leq i \leq n. \quad (45)$$

A formal approach with the Lagrange multipliers with 2 constraints yields the Lagrangian

$$\mathcal{L}(w, b, \alpha, \xi, \lambda) = \frac{1}{2}\|w\|^2 + C \sum_{1 \leq i \leq n} \xi_i - \sum_{1 \leq i \leq n} \alpha_i [y_i(w^T x_i + b) - 1 + \xi_i] - \sum_{1 \leq i \leq n} \lambda_i \xi_i \quad (46)$$

Take the partial derivative with respect to each variable considered yields the following equations from $\nabla \mathcal{L} = 0$

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{1 \leq i \leq n} \alpha_i y_i x_i = 0, \quad w = \sum_{1 \leq i \leq n} \alpha_i y_i x_i \quad (47)$$

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{1 \leq i \leq n} \alpha_i y_i = 0, \quad \sum_{1 \leq i \leq n} \alpha_i y_i = 0 \quad (48)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \lambda_i = 0, \quad \alpha_i + \lambda_i = C, \quad 0 \leq \alpha_i \leq C \quad (49)$$

The Dual Problem to the Primal Problem can be deduced by substituting the stationarity conditions back into the Lagrangian. Substituting $w = \sum_i \alpha_i y_i x_i$ and eliminating ξ_i via $\alpha_i + \lambda_i = C$, the Lagrangian reduces to

$$\mathcal{L} = \sum_{1 \leq i \leq n} \alpha_i - \frac{1}{2} \sum_{1 \leq i \leq n} \sum_{1 \leq j \leq n} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \quad (50)$$

which depends only on the dual variables α_i . Since we seek the tightest lower bound on the primal objective, the dual problem maximizes this expression subject to the constraints inherited from the stationary conditions

$$\max_{\alpha \geq 0} \inf_{w, b, \xi} \mathcal{L}(w, b, \xi, \alpha, \lambda) \quad (51)$$

Explicitly written as

$$\max_{\alpha} \sum_{1 \leq i \leq n} \alpha_i - \frac{1}{2} \sum_{1 \leq i \leq n} \sum_{1 \leq j \leq n} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \quad (52)$$

subject to

$$0 \leq \alpha_i \leq C, \quad 1 \leq i \leq n, \quad \sum_{1 \leq i \leq n} \alpha_i y_i = 0. \quad (53)$$

Once the optimal α_i^* are obtained, the primal weight vector is recovered as

$$w = \sum_{1 \leq i \leq n} \alpha_i^* y_i x_i \quad (54)$$

and the decision function with a kernel projection $K : X \times X \rightarrow \mathbb{R}^+$, where $K(x_i, x) = \phi(x_i)^T \phi(x)$, with $\phi : X \rightarrow \mathcal{F}$ for a new point x is given by

$$\hat{f}(x) = \text{sgn} \left(\sum_{1 \leq i \leq n} \alpha_i^* y_i \langle \phi(x_i), \phi(x) \rangle_{\mathcal{F}} + b \right). \quad (55)$$

Notably, only those x_i for which $\alpha_i^* > 0$ contribute to w and $\hat{f}(x)$. These are the support vectors, which lie on or within the margin. This sparsity is one of the central computational advantages of the SVM formulation as some α_j may be 0.

4. k-Means Clustering: Show that the k-means algorithm can be viewed as an iterative coordinate descent algorithm that minimizes the total within-cluster sum of squares (TWCSS). Prove that each assignment step and update step never increases the TWCSS objective. (5 points)

$$\text{TWCSS}(\mu, y) = \arg \min_{\mu, y} \left(\sum_{j=1}^k \sum_{i=1}^n y_{ij} \|x_i - \mu_j\|_2^2 \right) \quad (56)$$

where $y_{ij} = \mathbb{1}(x_i \in \text{Cluster}_j)$

We show that k-means alternately minimizes TWCSS over y and over μ , and that neither step increases the objective. We wish to minimize

$$\text{TWCSS}(\mu, y) = \sum_{j=1}^k \sum_{i=1}^n y_{ij} \|x_i - \mu_j\|_2^2, \quad y_{ij} = \mathbb{1}(x_i \in \text{Cluster}_j). \quad (57)$$

For fixed centroids, in this case the mean vector $\mu_1, \dots, \mu_k$, the objective decomposes over each point x_i independently, since each term $y_{ij} \|x_i - \mu_j\|_2^2$ depends only on the assignment of x_i . The constraint $\sum_j y_{ij} = 1$ means each point belongs to exactly one cluster, so we minimize by assigning each point to its nearest centroid first

$$y_{ij}^* = \mathbb{1} \left(j = \arg \min_{j'} \|x_i - \mu_{j'}\|_2^2 \right). \quad (58)$$

This is a coordinatewise minimization over y with μ held fixed. Since we replace each y_{ij} with its optimal value, the objective cannot increase

$$\text{TWCSS}(\mu, y^*) \leq \text{TWCSS}(\mu, y). \quad (59)$$

Now comes the second part of the coordinate descent. For fixed assignments y , the objective again decomposes over clusters. The contribution of cluster j is

$$\sum_{i=1}^n y_{ij} \|x_i - \mu_j\|_2^2. \quad (60)$$

This is a sum of squared Euclidean distances from μ_j to the points assigned to cluster j . Taking the gradient with respect to μ_j and setting it to zero

$$\nabla_{\mu_j} \left(\sum_{i=1}^n y_{ij} \|x_i - \mu_j\|_2^2 \right) = -2 \sum_{i=1}^n y_{ij} (x_i - \mu_j) = 0, \quad (61)$$

which yields the closed-form minimizer similar to the OLS estimator in regression, but in k - means the y_{ij} are like the binary weights.

$$\mu_j^* = \frac{\sum_{i=1}^n y_{ij} x_i}{\sum_{i=1}^n y_{ij}}, \quad (62)$$

Note that (62) is the mean of all points in cluster j . Since we replace each μ_j with its exact minimizer, the objective again cannot increase so we have

$$\text{TWCSS}(\mu^*, y) \leq \text{TWCSS}(\mu, y). \quad (63)$$

From above we have briefly seen that the k -means algorithm alternates between

$$y^{(t+1)} = \arg \min_y \text{TWCSS}(\mu^{(t)}, y), \quad \mu^{(t+1)} = \arg \min_{\mu} \text{TWCSS}(\mu, y^{(t+1)}), \quad (64)$$

which is exactly coordinate descent on the joint objective $\text{TWCSS}(\mu, y)$. We do this to avoid minimizing two variables jointly as that would lead to a NP hard problem. Also we can say that each step minimizes exactly over one block of variables while holding the other fixed, the sequence $\{\text{TWCSS}(\mu^{(t)}, y^{(t)})\}$ is monotonically decreasing

$$\text{TWCSS}(\mu^{(t+1)}, y^{(t+1)}) \leq \text{TWCSS}(\mu^{(t+1)}, y^{(t)}) \leq \text{TWCSS}(\mu^{(t)}, y^{(t)}). \quad (65)$$

Since the number of possible assignments y is finite as each of n points can be assigned to one of k clusters, giving at most k^n configurations, the algorithm must terminate in finite steps. Also note that usually the vector $\hat{\mu} \in \mathbb{R}^k$, $\hat{y} \in \mathbb{R}^n$ is initialized randomly in implementation, the function TWCSS may actually only be convex in one variable but is not jointly convex, so we can only guarantee that this algorithm converge to a local minimum.

Exercise 4: Video Summary & Self-Reflection (10 points)

Create a 5-minute video summarizing your work. *Video Link* : [Youtube Link](#)

References

- [1] R.A. Horn and C.R. Johnson. *Matrix Analysis*. Matrix Analysis. Cambridge University Press, 2013.

Iteration	LDA	QDA	NaiveBayes	9NN	SVM
1	0.1140		0.2847	0.2202	0.0697
2	0.1114		0.2886	0.2137	0.0749
3	0.0977	0.1739	0.2899	0.2202	0.0671
4	0.1166		0.2840	0.2176	0.0743
5	0.1192		0.2893	0.2098	0.0651
6	0.1088	0.1779	0.2938	0.2163	0.0704
7	0.1121	0.1629	0.2756	0.2163	0.0736
8	0.1257		0.2919	0.2235	0.0795
9	0.1049	0.1655	0.2990	0.2241	0.0736
10	0.1140		0.2821	0.2059	0.0827
11	0.1270	0.1609	0.2879	0.2300	0.0769
12	0.1121	0.1655	0.2717	0.2248	0.0736
13	0.1049	0.1661	0.3049	0.2085	0.0612
14	0.1173	0.1511	0.2619	0.2163	0.0769
15	0.1244		0.2938	0.2326	0.0788
16	0.1075		0.2971	0.2111	0.0697
17	0.1010		0.2723	0.2169	0.0691
18	0.1153	0.1700	0.3010	0.2176	0.0743
19	0.1094		0.3029	0.2098	0.0704
20	0.1283		0.3121	0.2208	0.0814
21	0.1153		0.2795	0.2261	0.0710
22	0.1166		0.2906	0.2156	0.0678
23	0.1166	0.1590	0.2775	0.2137	0.0704
24	0.1088	0.1616	0.2743	0.2378	0.0671
25	0.1127		0.3049	0.2241	0.0697
26	0.1134		0.2958	0.2130	0.0717
27	0.1023		0.2990	0.2111	0.0593
28	0.1153		0.2919	0.2326	0.0651
29	0.1055	0.1655	0.2945	0.2241	0.0756
30	0.1127		0.2971	0.2254	0.0651
31	0.1023		0.2906	0.2065	0.0651
32	0.1075	0.1779	0.3010	0.2182	0.0730
33	0.1127	0.1629	0.2801	0.2221	0.0678
34	0.1081		0.2984	0.2163	0.0762
35	0.1134	0.1596	0.2710	0.2085	0.0645
36	0.1016		0.2925	0.2130	0.0710
37	0.1244	0.1785	0.2964	0.2195	0.0717
38	0.1088		0.2814	0.2189	0.0678
39	0.1140	0.1720	0.2788	0.2104	0.0710
40	0.1107		0.2827	0.2195	0.0749
41	0.1140	0.1674	0.2736	0.2261	0.0684
42	0.1147	0.1785	0.2906	0.2150	0.0717
43	0.1173		0.2873	0.2261	0.0814
44	0.1055	0.1590	0.2756	0.2176	0.0704
45	0.1068	0.1831	0.2756	0.2326	0.0697
46	0.1127		0.3107	0.2046	0.0612
47	0.1121	0.1909	0.3010	0.2000	0.0808
48	0.0951	0.1733	0.3016	0.2065	0.0710
49	0.1277		0.3049	0.2137	0.0795
50	0.1055		0.2925	0.2306	0.0619

Table 3: Unadjusted - 50 Stratified Hold Outs with 57 Predictor Variables

Iteration	LDA	QDA	NaiveBayes	X9NN	SVM
1	0.2397	0.2423	0.2463	0.2384	0.1440
2	0.2599	0.2847	0.2984	0.2397	0.1609
3	0.2248	0.2515	0.2547	0.2280	0.1524
4	0.2404	0.2612	0.2645	0.2404	0.1537
5	0.2410	0.2736	0.2782	0.2358	0.1518
6	0.2417	0.2651	0.2645	0.2280	0.1472
7	0.2254	0.2827	0.2866	0.2508	0.1570
8	0.2476	0.2879	0.2893	0.2261	0.1531
9	0.2482	0.2619	0.2775	0.2404	0.1642
10	0.2352	0.2515	0.2593	0.2267	0.1505
11	0.2221	0.2521	0.2580	0.2267	0.1511
12	0.2664	0.2730	0.2697	0.2515	0.1583
13	0.2404	0.2788	0.2866	0.2228	0.1479
14	0.2215	0.2554	0.2599	0.2502	0.1511
15	0.2195	0.2651	0.2691	0.2476	0.1648
16	0.2541	0.2678	0.2658	0.2417	0.1511
17	0.2423	0.2866	0.2912	0.2495	0.1479
18	0.2684	0.2919	0.2951	0.2300	0.1603
19	0.2384	0.2534	0.2638	0.2423	0.1550
20	0.2684	0.2919	0.2945	0.2384	0.1603
21	0.2638	0.2906	0.2984	0.2319	0.1603
22	0.2182	0.2521	0.2528	0.2345	0.1446
23	0.2436	0.2599	0.2710	0.2515	0.1511
24	0.2573	0.2821	0.2847	0.2469	0.1564
25	0.2254	0.2638	0.2684	0.2358	0.1498
26	0.2554	0.2912	0.2990	0.2280	0.1498
27	0.2215	0.2469	0.2593	0.2293	0.1459
28	0.2208	0.2645	0.2612	0.2541	0.1479
29	0.2215	0.2664	0.2534	0.2521	0.1524
30	0.2704	0.2860	0.2788	0.2371	0.1479
31	0.2599	0.2827	0.2847	0.2241	0.1485
32	0.2495	0.2723	0.2788	0.2358	0.1694
33	0.2423	0.2619	0.2678	0.2417	0.1570
34	0.2430	0.2769	0.2678	0.2306	0.1440
35	0.2397	0.2873	0.2938	0.2463	0.1537
36	0.2163	0.2508	0.2593	0.2221	0.1440
37	0.2521	0.2795	0.2860	0.2371	0.1635
38	0.2450	0.2625	0.2697	0.2443	0.1629
39	0.2326	0.2580	0.2664	0.2306	0.1485
40	0.2423	0.2899	0.3003	0.2345	0.1518
41	0.2547	0.2769	0.2723	0.2469	0.1518
42	0.2651	0.2906	0.2971	0.2319	0.1616
43	0.2417	0.2899	0.2925	0.2430	0.1596
44	0.2378	0.2528	0.2658	0.2176	0.1453
45	0.2404	0.2554	0.2658	0.2306	0.1518
46	0.2397	0.2593	0.2593	0.2254	0.1570
47	0.2195	0.2456	0.2443	0.2430	0.1511
48	0.2547	0.2886	0.2984	0.2430	0.1616
49	0.2254	0.2658	0.2619	0.2352	0.1590
50	0.2319	0.2756	0.2834	0.2391	0.1590

Table 4: Adjusted 50 Stratified Hold Outs with 7 Predictor Variables